Project 1 - Image Server

Leah Weldon
21363670

1. Introduction

This project explores the creation of an embedded Domain-Specific Language (eDSL) for drawing graphical elements and integrates it into a web application. The objective was to design a language that enables users to define images using a set of basic shapes, transformations, and colour specifications, and then render these images through a web interface.

The web application itself is built using Haskell's Scotty and Blaze libraries to create a responsive interface that presents a gallery of pre-rendered images of shapes and translations. The images are generated from the eDSL using JuicyPixels, a Haskell library for pixel-based image rendering, which provides the functionality to render graphics as PNG files. The primary goals of this project included developing an eDSL capable of defining basic shapes (circle, rectangle, ellipse, polygon) and supporting affine transformations (scale, rotate, shear, translate), implementing colour management within the eDSL to allow each shape to be customised with unique colours, rendering images using the JuicyPixels library and presenting them within a Scotty-based web application and also providing descriptive DSL code snippets alongside each image to show the user the code structure used to create the images that are on display.

2. Design Choices

2.1 Choice of Basic Shapes

The eDSL includes four fundamental shapes which I implemented into the web application: Circle, Rectangle, Ellipse, and Polygon. These shapes were chosen for their versatility and simplicity, providing a broad range of visual options for the user while keeping the language easy to use and each of these shapes serves a different purpose:

- The circle offers a foundational rounded shape that is essential in many designs.
- The rectangle is essential for creating structured, square-like elements and serves as a base for composite shapes.
- The ellipse adds flexibility for more organic forms and varied aspect ratios.
- Finally the polygon allows for more complex shapes by defining closed convex forms with specified points. This also adds customization beyond simple geometric forms, enabling users to construct triangles, hexagons, or other polygonal shapes.

All of these shapes cover a wide range of common design elements.

2.2 Affine Transformations

The eDSL I created supports four affine transformations: Scale, Rotate, Shear, and Translate. These were chosen because they represent the most commonly used transformations in graphical design:

- Scale allows the user to adjust the size of a shape without altering its proportions, making it valuable for adapting shapes.
- Rotate enables the rotation of shapes around their origin which can add dynamism and variety to layouts.

- Shear creates an angular distortion which allows for perspective effects and skewed shapes.
- Translate shifts shapes along the X or Y axes, enabling precise positioning.

2.3 Colour Implementation

In this eDSL I have, colour is specified for each shape using an RGBA model, allowing customization of the Red, Green, Blue, and Alpha (transparency) channels. Each colour is set with values ranging from 0 to 255 which provides flexibility for solid fills as well as transparency effects. This straightforward RGB was chosen to keep the language intuitive and to allow for easy adjustments in shape appearance without too much complexity.

Using RGBA also opens possibilities for some layered effects, where transparency can be leveraged to overlay shapes and create richer compositions overall. This method supports both simple colour shapes and more visually complex designs with overlapping colours.

2.4 Relative Positioning of Shapes

The eDSL includes relative positioning functions too: over, left, right, below, and above. These functions allow shapes to be combined by positioning them in relation to each other without overlap:
- Over places one shape directly over another, useful for creating stacked or layered effects and having shape overlap or creating the illusion of different shapes.
- Left and right position shapes horizontally, useful for creating rows or left-to-right arrangements.
- Below and above position shapes vertically, allowing for columns or top-to-bottom designs.

These functions simplify the process of creating structured layouts by managing the alignment and positioning of elements and shapes, making it easier to build compositions without manually adjusting coordinates.

3. Implementation Details

3.1 Shape Representation and Transformations

Each shape is represented in the eDSL as a data structure within the Shape type. The shapes were defined as follows:

-Circle is represented by a centre point and an implicit radius.

-Rectangle is defined by an aspect ratio, allowing easy width and height adjustments.

-Ellipse is similar to the circle but allows distinct aspect ratios, supporting both horizontal or vertical elongation.

-Polygon is defined by a list of points, each point forming vertices in a clockwise order, allowing the creation of any convex polygon. I chose to do a triangle for this in my web application.

Each shape can be combined with transformations using a Transform data type. This type encapsulates each transformation as a composable unit:

-Scale accepts a scaling vector, allowing adjustments to width and height independently.

-Rotate uses a rotation matrix, calculated with trigonometric functions based on an angle in radians, allowing for flexible rotation of the shape.
-Shear applies a distortion along one axis, adding perspective effects.
-Translate shifts shapes on the X or Y axes, positioning them relative to other shapes or the canvas it is on.

Transformations are implemented as functions that alter the position and scale of points in each shape, applied using functional composition.

3.2 Combining Drawings

To combine multiple shapes into complex images, the eDSL provides positioning functions which are over, left, right, below, and above, all of which I have implemented into the web application. Each of these functions operates by adjusting shapes' positions relative to each other. Each function works by translating shapes using predefined offsets, which are calculated based on the dimensions of the involved shapes. For example, left shifts the left shape to the left of the right shape by a specified distance, ensuring that they do not overlap each other.

3.3 Using JuicyPixels for Rendering

The JuicyPixels library was used for rendering shapes as PNG images which are what can be seen in the web application. JuicyPixels provides a simple interface for creating images, making it suitable for rendering custom drawings generated by the eDSL. The rendering process follows these steps:

1. Pixel Mapping: Each shape is mapped to pixel coordinates within a Window, which defines the image's dimensions and boundaries.
2. Colour Application: The colour for each pixel is determined based on whether it falls inside a shape or in the background.
3. Image Generation: The render and rgbRender functions generate grayscale or colour images. These functions apply transformations, calculate pixel positions, and set RGB or RGBA values.

JuicyPixels was chosen for its efficiency and ease of use in functional programming, aligning well with Haskell's structure.

3.4 Scotty and Blaze Integration

To display the generated images, the eDSL integrates Scotty and Blaze which are two Haskell libraries for building web applications:
-Scotty handles HTTP routing and serves the web pages, providing endpoints for each image and the CSS stylesheet which I added to make it more visually appealing. For instance, routes like /circle or /rectangle serve specific images generated by the eDSL.
-Blaze is used to generate HTML content, making it easy to create a web page layout and integrate DSL code alongside rendered images. Using this, each image is linked on the web page with a corresponding description and code snippet that generated it so that the user can see how the image they are viewing was made.

4. Sample Images and DSL Code

The following examples outline some of the sample images generated by the DSL, with some accompanying descriptions of the code that produced them. I have provided these examples to demonstrate how the DSL manages shapes, transformations, colours, and positioning.

- Circle Over Rectangle: The DSL code *over [(scale (point 1.0 1.0), circle, createColor 255 0 255 255)] [(identity, rectangle 2 1, createColor 0 0 255 255)]* places a circle over a rectangle. This layered effect shows the "over" function, which stacks one shape directly on top of another without overlapping edges.

- Polygon Left of Rectangle: Using the code *left [(identity, polygon [point (-2) 0, point 2 0, point 0 2], createColor 0 255 0 255)] [(identity, rectangle 2 1, createColor 0 0 255 255)]*, positions a polygon to the left of a rectangle. This demonstrates the DSL's relative positioning functions, allowing shapes to be placed side-by-side without overlap.

- Translated Circle: The code *[(translate (point (-6) 1), circle, createColor 255 165 0 255)]* applies a translation to a circle, shifting it to the left. This example highlights the DSL's translate transformation, showing how shapes can be repositioned accurately within the coordinate space.

- Rotated Ellipse: The rotated ellipse, created using the code *[(rotate 30, ellipse 2 1, createColor 128 0 128 255)]*, demonstrates how the rotate function enables shapes to be oriented at different angles.

Each of these examples shows the core functionality of the DSL, constructing shapes, applying transformations, assigning colours, and arranging elements spatially.

5. Optimizations

In designing this DSL, optimizations were focused on creating a clear and modular structure that simplified the integration of various transformations and shape compositions. Rather than implementing complex computational optimizations, I placed emphasis on organising the code in a way that improved readability and maintainability, which is beneficial for further development and was beneficial when I was debugging my code.

Each transformation was defined as an independent function, allowing for easy modification and layering. By organising transformations like this, the DSL can support complex compositions in a straightforward manner. This approach ensured that transformations could be combined flexibly without creating excessive dependencies between functions, making the code easier to extend or modify if additional transformations or features are added in the future.

Structuring relative positioning functions to operate independently helped streamline the code and prevent overlap to ensure shapes are positioned consistently. This design choice was beneficial in managing how shapes interacted without requiring complex computations to resolve overlaps or alignment issues.

While these structural optimizations do not directly impact runtime performance, they enhance the project's maintainability and adaptability, making it easier to build upon or refactor the DSL code if needed.

6. Reflection

Designing this eDSL presented a few challenges, particularly in balancing simplicity with functionality. One of the primary challenges was defining a flexible but also straightforward structure for shape transformations and positioning. Integrating relative positioning functions such as over, left, and below highlighted the need for precise handling of coordinates to maintain alignment and I needed to ensure that both of the shapes could still be seen in the image.

Some key takeaways from this project for me include the value of modular design in DSL development and also the importance of clear function definitions for some of the more complex tasks. When I focused on modularity, the eDSL's code became easier to debug, extend, and adapt for when I was progressing through all of the assignment specifications. Integrating Scotty and Blaze also highlighted to me the potential of Haskell's web eDSLs to create web-based interfaces for non-interactive applications. The combination of these frameworks with JuicyPixels allowed for the rendering of images in a web format, helping me to fulfil the requirements of the assignment and demonstrate the eDSL's capabilities.

Some potential improvements to the project could involve enhancing interactivity, allowing users to input custom shapes or transformations into the web application. Adding support for scalable vector graphics instead of static PNG rendering would also improve the resolution and flexibility of generated images. Overall, these enhancements could broaden the eDSL's utility and appeal for a wider range of applications.

7. Conclusion

With this project I was able to successfully demonstrate the creation and application of a custom drawing eDSL integrated with Haskell's Scotty and Blaze libraries to deliver a web-based rendering solution. The eDSL supports essential shapes such as circles, rectangles, ellipses, and polygons, along with key affine transformations, including scaling, rotation, shearing, and translation. These foundational elements allow for flexible image construction, and the integration of JuicyPixels enables efficient rendering into PNG format, meeting the project requirements.

Through this project, the eDSL showcases a functional design that meets the needs of both web rendering and some shape manipulation. The addition of relative positioning functions like over, left, and above supports complex shape compositions, enhancing the versatility of the eDSL for more creative layouts. This application provides a solid foundation for future work, such as user interactivity or SVG support. Ultimately, the project highlights the effectiveness of using Haskell

to develop a specialised eDSL and offers a clear example of how web frameworks can be leveraged to display programmatically generated graphics.